Practical Week 12

Session 12:

Task1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.



```
koumudi@Koumudi: ~/OSSP/
 GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5
#define ITEMS 100000

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    long long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        sum += buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    printf("Consumer processed %d items.\n", ITEMS);
    printf("Consumer sum: %lld\n", sum);

    return NULL;
}

int main()
{
    pthread_t producer_thread;
```

```
    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    struct timespec start, end;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Buffer size: %d\n", BUFFER_SIZE);
    printf("Execution time: %.6f seconds\n", elapsed);
    printf("Throughput: %.2f items/second\n",
           ITEMS / elapsed);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/
koumudi@Koumudi:~$ cd ~/OSSP/PRACTICAL
mkdir Session_12
cd Session_12
pwd
/home/koumudi/OSSP/PRACTICAL/Session_12
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ nano producer_consumer.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ gcc producer_consumer.c -o producer_consumer -pthread
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ./producer_consumer
Consumer processed 100000 items.
Consumer sum: 4999950000
Buffer size: 5
Execution time: 0.874887 seconds
Throughput: 114300.43 items/second
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ nano producer_consumer.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ gcc producer_consumer.c -o producer_consumer -pthread
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ./producer_consumer
Consumer processed 100000 items.
Consumer sum: 4999950000
Buffer size: 2
Execution time: 2.157799 seconds
Throughput: 46343.52 items/second
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$
```

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    sleep(3);

    printf("\nDeadlock scenario demonstrated.\n");
    printf("Both threads are waiting for a resource held by the other.\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ nano deadlock.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ls -l deadlock.c
-rw-r--r-- 1 koumudi koumudi 1182 Sep 24 16:59 deadlock.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ gcc deadlock.c -o deadlock -pthread
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ gcc deadlock.c -o deadlock -pthread
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ./deadlock
Thread 2: locking Resource 2
Thread 1: locking Resource 1
Thread 2: waiting for Resource 1
Thread 1: waiting for Resource 2

Deadlock scenario demonstrated.
Both threads are waiting for a resource held by the other.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$
```

Task2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    sleep(3);

    printf("\nDeadlock scenario demonstrated.\n");
    printf("Both threads are waiting for a resource held by the other.\n");

    return 0;
}
```

```
Deadlock scenario demonstrated.
Both threads are waiting for a resource held by the other.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ nano deadlock.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ nano deadlock.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ gcc deadlock.c -o deadlock -pthread
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ./deadlock
Thread 2: locking Resource 2
Thread 1: locking Resource 1
Thread 2: waiting for Resource 1
Thread 1: waiting for Resource 2

Deadlock scenario demonstrated.
Both threads are waiting for a resource held by the other.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$
```

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void lock_resources_in_order()
{
    pthread_mutex_lock(&resource1);
    printf("Locked Resource 1\n");

    pthread_mutex_lock(&resource2);
    printf("Locked Resource 2\n");
}

void unlock_resources()
{
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
}

void *thread1(void *arg)
{
    printf("Thread 1 starting...\n");

    lock_resources_in_order();

    printf("Thread 1 acquired both resources\n");
    sleep(1);

    unlock_resources();

    printf("Thread 1 released both resources\n");

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2 starting...\n");

    lock_resources_in_order();

    printf("Thread 2 acquired both resources\n");
    sleep(1);

    unlock_resources();

    printf("Thread 2 released both resources\n");

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nDeadlock prevented using resource ordering.\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ nano deadlock_prevention.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ./deadlock_prevention
Thread 1 starting...
Locked Resource 1
Locked Resource 2
Thread 1 acquired both resources
Thread 2 starting...
Thread 1 released both resources
Locked Resource 1
Locked Resource 2
Thread 2 acquired both resources
Thread 2 released both resources

Deadlock prevented using resource ordering.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$
```

```
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$ ls -lh
total 72K
-rwxr-xr-x 1 koumudi koumudi  16K Sep 24 17:08 deadlock
-rw-r--r-- 1 koumudi koumudi 1.2K Sep 24 17:07 deadlock.c
-rwxr-xr-x 1 koumudi koumudi  17K Sep 24 17:12 deadlock_prevention
-rw-r--r-- 1 koumudi koumudi 1.3K Sep 24 17:10 deadlock_prevention.c
-rw-r--r-- 1 koumudi koumudi 1.2K Sep 24 16:58 k
-rwxr-xr-x 1 koumudi koumudi  17K Sep 24 16:49 producer_consumer
-rw-r--r-- 1 koumudi koumudi 1.9K Sep 24 16:48 producer_consumer.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_12$
```